

Scaling Law 的核心意思是：大模型的性能提升不是完全玄学，而是在很多情况下会随着数据量、参数量、算力的增加，呈现出相当稳定、可预测的规律。

通俗说：

以前训练大模型像“烧钱试错”，不知道模型要多大、数据要多少、训练多久、用什么结构更好；Scaling Law 想解决的问题是：**能不能先用小模型做实验，再推算大模型会怎样，从而少走弯路。**

1. 为什么 Scaling Law 重要：它是“大模型工程的省钱公式”

讲义开头提出的问题是：**调超参数太贵了**。比如优化器、模型深度、参数规模、数据规模、训练步数，这些如果都在大模型上试，会消耗巨量算力。

讲义给出的新思路是：

不要直接在大模型上乱试，而是先训练一些小模型，观察规律，再外推到大模型。

通俗解释：

这就像你不可能为了测试一座桥的承重，每次都造一座完整大桥。更合理的做法是先做缩小模型，找到材料、结构和承重之间的规律，再推算真实大桥的表现。

在大模型里也是一样：

先训练 100M、300M、1B 规模的小模型，看 loss 怎么下降，然后预测 10B、100B 模型大概会到什么水平。

2. Scaling Law 最经典的形式：log-log 图上接近一条直线

讲义反复强调一个核心现象：

Loss 和数据规模、参数规模、算力之间，经常在 log-log 图上呈现近似线性关系。

这句话听起来抽象，换成大白话：

假设你把数据量从 1 倍增加到 10 倍，loss 会下降一截；

再从 10 倍增加到 100 倍，loss 又下降一截；

每次“按倍数扩大”带来的收益，往往比较稳定，但收益会逐渐变小。

所以它不是说：

数据多一点，模型马上变强很多。

而是说：

在足够大的尺度上，模型性能会按某种稳定斜率持续改善。

这也是为什么很多大模型公司愿意持续堆数据、堆参数、堆算力：因为从经验上看，**只要方向对，scale 确实经常能换来性能提升**。

3. 数据 Scaling Law：数据越多，误差通常越低

讲义第一部分讲的是 **Data vs Performance**：数据量和模型表现之间的关系。它指出，数据 scaling law 可以理解为：

一个把数据集大小 n 映射到 error/loss 的简单公式。

最直观的表达是：

数据越多，模型越不容易犯错，但提升速度不是线性的。

通俗解释：

你学英语时，背 100 个单词到 1000 个单词，提升很明显；
但从 10000 个单词到 11000 个单词，提升可能就没那么明显。
大模型也是类似：数据越多越好，但边际收益递减。

讲义用“估计均值”做了一个玩具例子：

如果你要估计一个总体均值，样本越多，估计误差越小，而且误差大致会按某种多项式速度下降。

这就是 scaling law 的理论直觉：

学习误差天然会随着样本数量增加而下降，而且很多时候下降规律可以近似成幂律。

4. 为什么是“幂律”：因为学习本质上是在降低估计误差

讲义里说，很多 scaling law 可以理解为：

Estimation error naturally decays polynomially.
估计误差会自然地以多项式速度衰减。

通俗解释：

模型训练其实是在用有限样本去估计真实世界的规律。样本越多，估计越准。

但这个“变准”的速度不是无限快的，而是有一个比较稳定的衰减速度。

比如最简单的均值估计里，误差大致和样本量的倒数有关。
样本越多，误差越小，但不是样本翻倍、错误就减半这么简单。

讲义进一步提出一个更深的观点：

Scaling law 的斜率可能和数据的“内在维度”有关。

大白话说：

如果一个任务本身很复杂，需要理解的隐藏因素很多，那么它的学习曲线就可能下降得更慢；
如果任务比较简单，数据增加后性能可能提升得更快。

5. 数据组成也重要：不只是“数据越多越好”

讲义还讲了一个高级问题：**distribution shift**，也就是训练数据和测试数据分布不完全一致时怎么办。

它提到一个结论：

数据组成会影响 scaling law 的 offset，而不一定改变 slope。

这句话可以这样理解：

假设两个模型都随着数据增加而变好，改善速度差不多；
但是如果一个模型的数据质量更高、覆盖更全面，它的整条曲线可能整体更低，也就是同样数据量下 loss 更低。

通俗说：

数据质量和多样性决定了你起跑线在哪里；
数据规模决定了你沿着这条线能往前走多远。

这对大模型很关键。不是说随便爬更多网页就一定最好，而是要考虑数据分布、数据多样性、少数群体、公平性，以及目标任务是否被覆盖。

讲义还提到 fairness：少数群体上的表现也可能遵循 scaling law，因此可以用 scaling law 来指导更公平的数据采集。

6. 模型 Scaling Law：参数越多，通常 loss 越低

第二部分进入模型工程问题：
怎么高效设计大语言模型？

讲义讨论了几个典型问题：

1. Transformer 是否比 LSTM 更好？
2. Adam 是否比 SGD 更好？
3. 模型应该更深还是更宽？
4. 有限算力下，是训练更大的模型，还是训练更久？
5. 是收集更多数据，还是买更多 GPU？

Scaling law 的价值在于：

这些问题不一定要在最大模型上直接试，可以先在小模型上建立趋势，再预测大模型表现。

通俗解释：

你要开一家连锁店，不会一开始就在全中国开 1000 家店试错。你会先开几家样板店，看不同选址、装修、定价、菜单组合的效果，再决定怎么扩张。

大模型训练也是一样：

先用小模型试不同结构和超参数，看趋势，再决定大模型怎么配。

7. 架构选择：Transformer vs LSTM 可以通过小模型趋势预测

讲义举了 Transformer 和 LSTM 的对比。暴力方法是花几千万美元训练一个 LSTM 版 GPT-3，看它到底行不行。Scaling law 方法则是：

训练较小规模的 Transformer 和 LSTM，比较它们的 scaling 曲线，然后推测大规模时谁更好。

通俗解释：

如果两条曲线在小规模时已经显示出 Transformer 更低 loss、更好斜率，那就没有必要真的烧钱训练一个超大 LSTM 来证明它不行。

这说明 Scaling Law 不是纯理论，它直接影响模型架构决策。

8. 超参数也能被预测：优化器、深度、架构都可以提前判断

讲义有一个很重要的 takeaway：

大模型上的超参数效果，某种程度上可以在训练前通过小模型预测。

包括：

- optimizer choice：Adam vs SGD；

- model depth：模型层数；
- architecture choice：架构选择。

讲义给出的 scaling-law-based design procedure 是：

1. 训练几个小模型；
2. 建立 scaling law；
3. 根据 scaling law 预测大模型上哪个超参数更优。

通俗解释：

不是“大模型训练完才知道这个设计好不好”，而是“训练之前就用小实验判断大方向”。

这对大模型公司非常重要，因为一次大模型训练失败，可能就是几百万甚至上千万美元的成本。

9. 数据量和模型大小要一起看：小模型吃太多数据会浪费

讲义专门讲了 **Model-data joint scaling**：模型大小和数据大小的联合关系。

核心问题是：

我们到底需要更多数据，还是更大的模型？

讲义指出：

很多数据喂给太小的模型，可能是浪费。因为小模型容量有限，吃再多数据也学不下更多复杂规律。

通俗解释：

你让一个小学生读 1000 本专业论文，他不一定能吸收；

不是论文不够多，而是他的认知容量不够。

模型也是一样：模型太小，数据再多也会出现容量瓶颈。

所以大模型训练要平衡：

- 参数量；
- 数据 token 数；
- 训练计算量；
- 训练步数。

这也是后来很多大模型训练策略的基础：**模型不能只大，数据也要匹配；数据不能只多，模型容量也要跟上。**

10. 算力 tradeoff：固定算力下，大模型“适度欠训练”可能更好

讲义里有一个很有意思的结论：

在固定 compute budget 下，properly undertrained models are better。

也就是：

在一定条件下，与其训练一个小模型训练得很充分，不如训练一个更大的模型，即使它还没有完全训练到收敛。

通俗解释：

假设你只有固定时间准备考试。

方案 A：反复刷小学题，刷到满分；

方案 B：开始刷更高阶的题，虽然还没完全掌握，但能力上限更高。

对大模型来说，后者可能更划算。

这也是 GPT-3 时代的一个重要思想：

宁愿训练一个很大的模型，即使它没有完全吃透所有数据，也可能比小模型充分训练更强。

不过这里要注意，这份讲义主要反映的是 Kaplan 等早期 scaling law 的观点。后来 Chinchilla 等研究进一步强调了“参数量和数据量更均衡”的重要性。也就是说，这个结论在历史上很重要，但不能机械理解成“模型越大越好、欠训练越多越好”。

11. 不是所有参数都一样：embedding 参数和非 embedding 参数效果不同

讲义还提醒：我们经常说“参数量”，但不是所有参数都同等重要。它特别提到：

Embedding layer parameters don't behave the same.

通俗解释：

模型参数就像公司员工人数，但不是所有员工对产出贡献一样。

研发、销售、行政、客服都算员工，但增加不同岗位的人，对业务增长的作用不同。

大模型里也是：

embedding 层参数、attention 层参数、MLP 层参数，对 scaling 的贡献可能不同。

所以单纯看“总参数量”有时会误导，要看有效参数量、架构分布和计算路径。

这也和后来的 MoE、稀疏模型有关：

模型总参数很多，但每次推理只激活一部分参数，所以“总参数量”和“实际计算量”之间不再是一回事。

12. Scaling Law 还能预测未来能力：哪些任务能靠 scale 暴力解决？

讲义最后一部分讨论 forecasting：

能不能用 scaling law 预测某些 benchmark 未来会不会被解决？

它举了几个例子：

- Winograd Schema；
- SAT analogies；
- Word in Context；
- 算术任务中的 phase transition。

有些任务随着模型规模扩大，表现持续改善。

比如 Winograd、SAT analogies 这类任务，图上似乎有比较清晰的提升趋势。

但有些任务改善很慢，比如 Word in Context，讲义称其 scaling 接近于零。

通俗解释：

不是所有问题都能靠“堆模型、堆数据、堆算力”自然解决。

有些题像爬坡：越大越好，持续进步；

有些题像撞墙：模型变大了，但能力没怎么变；

还有些题像突然开窍：一开始没变化，到某个规模突然跃迁。

13. Phase Transition：能力可能突然涌现

讲义最后提到一个大模型领域非常重要的问题：**phase transitions**，也就是性能的突然跃迁。

此前大部分 scaling law 是连续线性趋势，但 GPT-3 论文里观察到一些任务会出现突然的、不连续的性能跳跃。

通俗解释：

有些能力不是一点点变强，而是模型到某个规模之后突然“会了”。

比如小模型做算术一直很差，但参数量、数据量、训练规模到达某个阶段后，突然开始做对一部分题。

这类现象后来也常被称为 emergent abilities，涌现能力。

但讲义也很谨慎：

这可能是语言模型 scaling 里最大的未知问题之一。

换句话说：

Scaling law 很有用，但它不能解释所有事情，尤其不能完全预测“什么时候突然涌现新能力”。

14. 这份讲义的三个核心结论

结论一：Data scaling

数据规模和误差之间存在非常稳定的关系。

数据越多，通常 loss 越低，而且在 log-log 图上经常接近直线。

大白话：

多喂数据通常有用，而且这种“有用”不是完全随机的，是可以预测的。

结论二：Model scaling

模型参数量、架构选择、优化器、深度等，也可以通过 scaling law 进行工程决策。

大白话：

不要靠玄学设计大模型，要用小模型实验推断大模型表现。

结论三：Scaling as prediction

Scaling law 可以用来预测哪些任务可能通过扩大规模解决，哪些任务可能不行。

大白话：

有些问题可以靠堆算力 brute force，有些问题不能；Scaling Law 帮我们判断值不值得继续堆。

15. 总结

Scaling Law 是大模型训练中的经验规律和工程决策工具。它通过小规模实验拟合数据量、参数量、算力和 loss/benchmark performance 之间的关系，从而帮助团队在正式训练大模型前判断架构选择、超参数、数据规模和算力分配。它的价值是降低试错成本，但它也有边界：不同任务的 scaling slope 不同，有些任务可能出现 phase transition，有些任务则几乎不随 scale 改善。

Scaling Law 的本质不是“盲目堆大”，而是“用可预测的经验规律指导 scale”。它回答的是：在有限预算下，数据、参数和算力应该怎么配，哪些能力可能靠规模提升获得，哪些问题可能需要新的数据、架构或训练方法。